

included for clarity in the explanation).

```
1. from sage.graphs.graph import Graph

2. adj_list = {
    "A": {"B": 6, "D": 7, "F": 3},
    "B": {"A": 6, "C": 5, "E": 9},
    "C": {"B": 5, "D": 8, "E": 4},
    "D": {"A": 7, "C": 8, "E": 1},
    "E": {"B": 9, "C": 4, "D": 1,
    "F": 2},
    "F": {"A": 3, "E": 2}
}

3. sgraph = Graph(adj_list,
weighted=True)

4. sgraph.show(edge_labels=True)
```

Now, let us go through the code line by line to understand how it works. Line 1 imports the *Graph* class from the *sage.graphs.graph* module. This class is used to create and work with graph objects in SageMath. Line 2 defines an adjacency list. This line creates a Python dictionary called *adj_list* that represents the structure of a weighted graph. The structure of the dictionary is as follows: the keys are the vertex labels like 'A', 'B', 'C', etc, and the values are dictionaries containing adjacent vertices and their corresponding edge weights. For example, "A": {"B": 6, "D": 7, "F": 3} means vertex A is connected to vertices B, D, and F with weights 6, 7, and 3, respectively. Line 3 creates a weighted graph called *sgraph* using the *Graph* class. The adjacency list *adj_list* is given as a parameter to construct the graph. The parameter *weighted* is set to *True* to specify that the graph is weighted, meaning edges have numerical values associated with them. Finally, Line 4 displays a visual representation of the graph *sgraph*. The parameter *edge_labels* is set to *True* to include edge weights as labels on the edges.

Now, let us execute the code to view the output. In the first article in this series, we explored three different ways to execute SageMath code. We

have decided to use an online tool called CoCalc in the initial articles in this series, so that new users can immediately start using SageMath without any installation. Log in to the CoCalc platform, type the code in the window, and save it as *graph1.sagews*. Now, press the *Run* button to execute the code. Figure 2 shows the graph generated by the above code on execution.

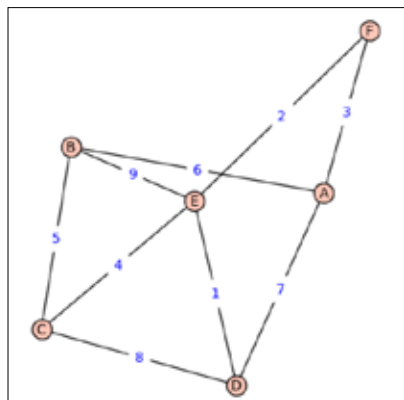


Figure 2: *Graph_1* generated by SageMath

Now, here is a surprise. Compare Figures 1 and 2; they don't look the same. I promised to generate *Graph_1* (as shown in Figure 1) using SageMath, and the graph obtained (as shown in Figure 2) looks different. What happened? Well, this is an important aspect of graphs. The same graph can be drawn or embedded in different ways on paper. Thus, the two graphs in Figures 1 and 2 are equivalent; more technically, these two graphs are isomorphic, even though they look different on paper. However, the essential properties of these two graphs are the same. They have the same set of vertices, the same number of edges, and the edges connect the same pair of vertices. Furthermore, the edge weights also remain the same. For example, the edge connecting vertices A and B has a weight of 6 in both representations.

I encourage you to take some time to verify that the two representations of the graph have the same number of vertices, edges, connectivity, and

weights. For a further illustration of this fact, check the Wikipedia article on the Petersen graph. The Petersen graph, comprising 10 vertices and 15 edges, is a well-known example in graph theory that illustrates how a single graph can be visually represented in various ways. Despite having the same underlying structure, different drawings or embeddings can highlight distinct features. Whether it is presented in a circular arrangement, a star-like pattern, or any other layout, the connectivity and relationships among vertices remain constant for the Petersen graph.

Recall that the above code creates a weighted graph object in SageMath using an adjacency list representation. However, this is not the only way to represent graphs in SageMath (or, for that matter, in general). One such method is to use an adjacency matrix (a grid of numbers). Add the following line of code at the end of *graph1.sagews* and execute it again.

```
print(sgraph.adjacency_matrix( ))
```

On execution, the following additional output will be printed on the screen. But what does it mean?

```
[0 1 0 1 0 1]
[1 0 1 0 1 0]
[0 1 0 1 1 0]
[1 0 1 0 1 0]
[0 1 1 1 0 1]
[1 0 0 0 1 0]
```

Figure 3 will help us understand the working of an adjacency matrix. An adjacency matrix is a matrix in which the rows and columns correspond to the vertices of the graph. For *Graph_1*, the vertices are A to F, corresponding to the six people Amrita, Biju, Chitra, Dev, Ekta, and Farhan; hence the 6 x 6 matrix. An adjacency matrix has an entry of 1 in the corresponding cell where a row and column intersect if and only if there is an edge between the pair